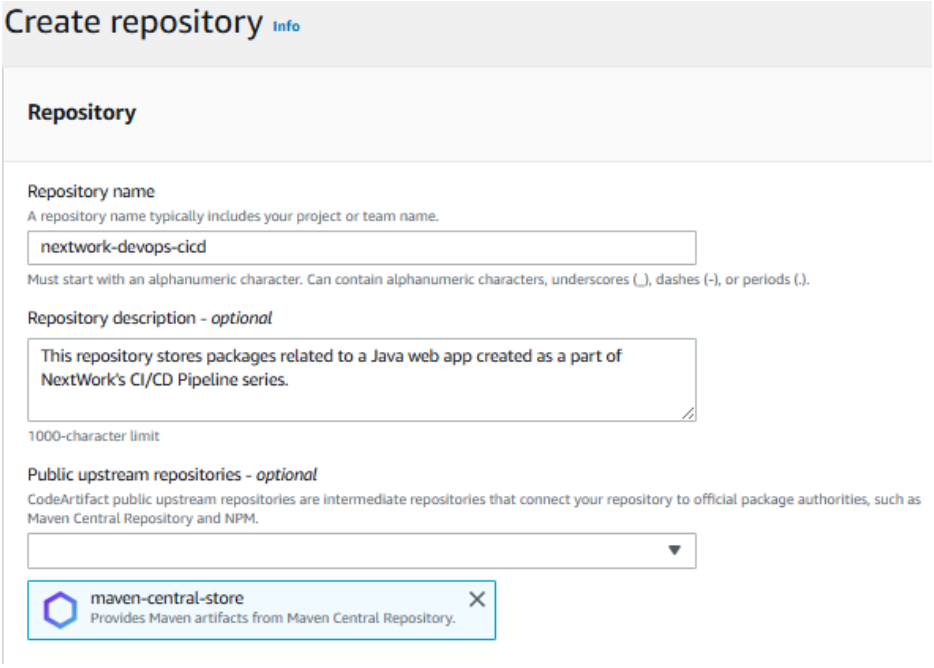
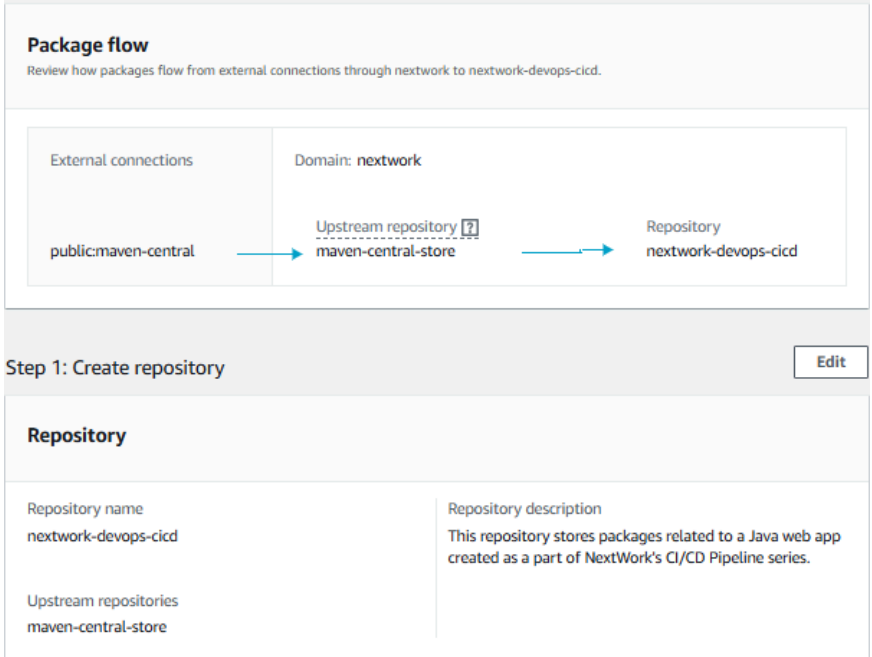
AWS CI/CD Pipeline Pt2

Create and configure your CodeArtifact repository

- Head back to the AWS Management Console.
- Head to the CodeArtifact service.
- In the CodeArtifact console, in the left-hand menu, click on **Repositories**.
- Click the **Create repository** button to start creating a new repository.
- On the **Create repository** page, head to the **Repository configuration** section.
- In the **Repository name** field, enter nextwork-devops-cicd
- Under description, **This repository stores packages related to a Java web app created as a part of NextWork's CI/CD Pipeline series.**
- Under **Public upstream repositories - optional**, select the checkbox next to **maven-central-store**.
- This will configure **Maven Central** as an upstream repository for your CodeArtifact repository.



- Under **Domain selection**, choose **This AWS account**.
- Under **Domain name**, enter nextwork
- Now we're on the last page! Let's **Review and create**.
- Review the details of your repository configuration, including the package flow at the top.



Create an IAM Policy for CodeArtifact Access

For Maven to start working with CodeArtifact, we need to create an **IAM role** that grants our **EC2 instance** the permission it needs to access CodeArtifact

Get CodeArtifact connection instructions

- On your newly created repository's page, click the **View connection instructions** button at the top right corner.
- In the **Connection instructions** page, we're configuring how Maven will connect to your CodeArtifact repository.
- For **Operating system**, select **Mac and Linux**.
- For **Package manager client**, select **mvn** (Maven)
- Make sure that **Configuration method** is set to **Pull from your repository**.
- Nice! The menu will now show you the steps and commands needed to connect Maven to your CodeArtifact repository.

Export CodeArtifact authorization token

- In the **Connection instructions** dialog, find **Step 3: Export a CodeArtifact authorization token...**.
- Copy the entire command in **Step 3**.
- Go back to your VS Code terminal, which is connected to your EC2 instance.

- Paste the copied command into the terminal and press **Enter** to run it
- We get an error because the EC2 doesnt have permission to access other AWS resources

Create a new IAM policy

- In the AWS Management Console, head to the the IAM console.
- In the IAM console, in the left-hand menu, click on **Policies**.
- Click the **Create policy** button to start creating a new IAM policy
- On the **Create policy** page, select the **JSON** tab.
- Replace the default content in the text editor with the following JSON policy document. Copy and paste the entire JSON code block

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "codeartifact:GetAuthorizationToken",
                "codeartifact:GetRepositoryEndpoint",
                "codeartifact:ReadFromRepository"
            ],
            "Resource": "*"
        },
        {
            "Effect": "Allow",
            "Action": "sts:GetServiceBearerToken",
            "Resource": "*",
            "Condition": {
                "StringEquals": {
                    "sts:AWSServiceName": "codeartifact.amazonaws.com"
                }
            }
        }
    ]
}
```

- On the **Review policy** page, in the **Policy name** field, enter codeartifact-nextwork-consumer-policy.
- In the **Description - optional** field, add a description like: Provides permissions to read from CodeArtifact. Created as a part of NextWork CICD Pipeline series.
- Click the **Create policy** button to create the IAM policy

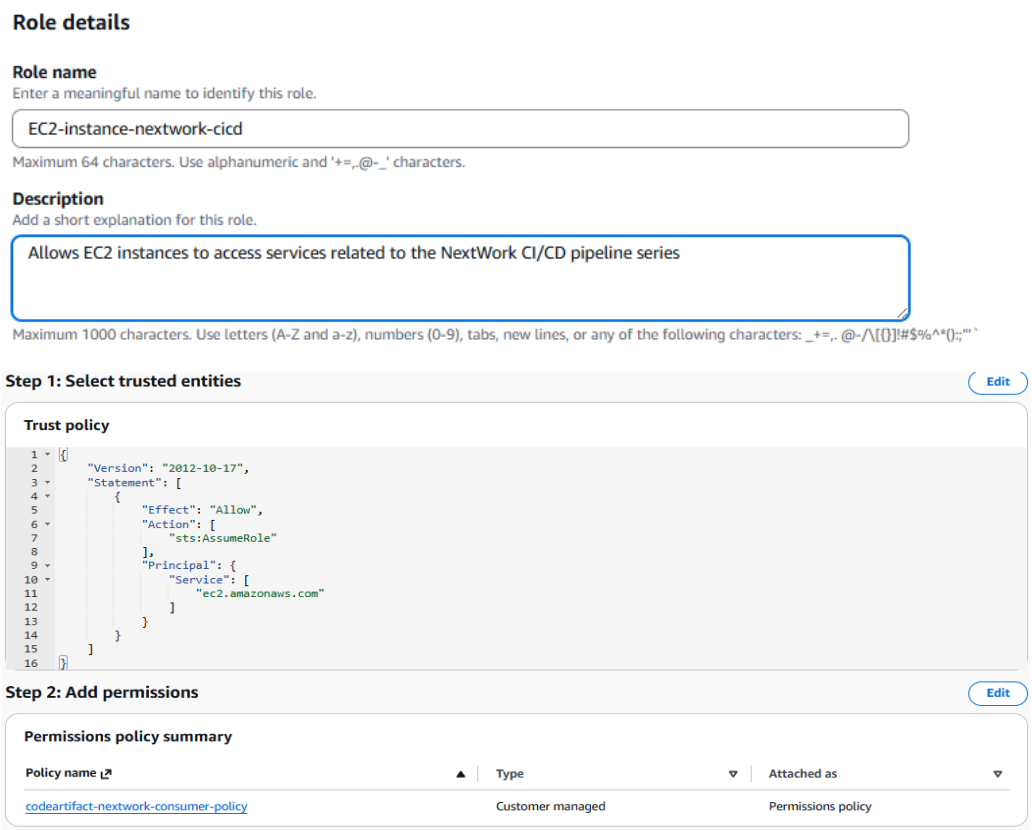
Attach IAM Policy and Verify CodeArtifact Connection

Create a new IAM role for EC2

- In the IAM console, in the left-hand menu, click on **Roles**.
- Click the **Create role** button to start creating a new IAM role.
- For **Select entity type**, choose **AWS service**.
- Under **Choose a use case**, select **EC2** from the list of services
- Click **Next** to proceed to the **Add permissions** step
- In the **Add permissions** step, in the **Filter policies** search box, type *codeartifact-nextwork-consumer-policy*.
- Select the checkbox next to the *codeartifact-nextwork-consumer-policy* that you created in the previous step.
- Click **Next** to head to the **Name, review, and create** step.

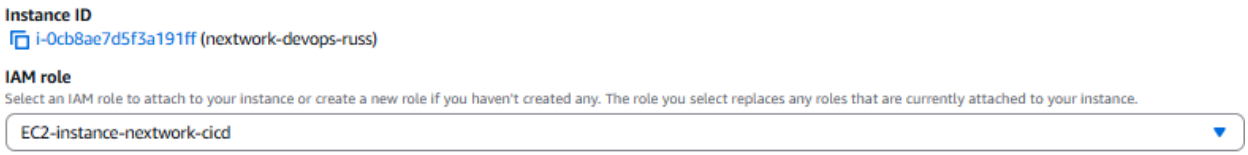
In the **Name, review, and create** step:

- In the **Role name** field, enter EC2-instance-nextwork-cicd.
- In the **Description - optional** field, enter:
  Allows EC2 instances to access services related to the NextWork CI/CD pipeline series.
- Next, in the review page, click the **Create role** button to create the IAM role



Attach IAM role to EC2

- Now, we need to associate this IAM role with your EC2 instance.
- Head back to the EC2 console.
- Head to the Instances tab
- Select your running EC2 instance (nextwork-devops-yourname)
- Click on **Actions** in the menu bar, then select **Security**, and then **Modify IAM role**.
- In the **Modify IAM role** dialog box, select **Refresh** to load the IAM roles available for your EC2 instance.
- Under **IAM role**, select the IAM role you just created, EC2-instance-nextwork-cicd, from the dropdown menu.
-



Re-run export token command

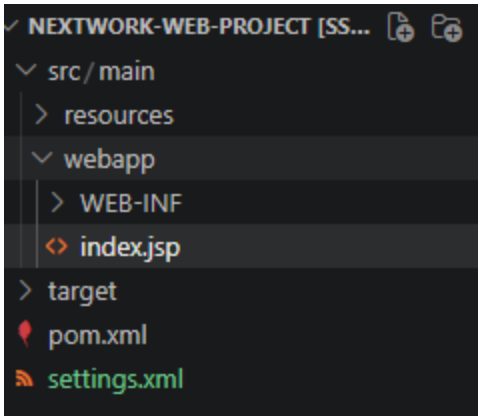
Now that your EC2 instance has the necessary IAM role attached, let's re-run the command to export the CodeArtifact authorization token.

This time, your EC2 instance should be able to retrieve the token, since it has the necessary permissions from the IAM role.

- Head back to your VS Code terminal connected to your EC2 instance.
- Re-run the same export token command from Step 3

See Packages in CodeArtifact

- In VS Code, in your left hand **file explorer**, head to the root directory of your nextwork-web-project.
- Create a new file at the root of your **nextwork-web-project** directory
- Name the new file **settings.xml**



- Open the settings.xml file. If you created a new file, it will be empty.
- In your settings.xml file, add the **<settings>** root tag if it's not already there
- Go back to the CodeArtifact connection settings panel.
- From the **Connection instructions** dialog, copy the XML code snippet from **Step 4: Add your server to the list of servers in your settings.xml**
- Paste the code in the **settings.xml** file, in between the <settings> tags
- Let's copy the XML code snippet from **Step 5: Add a profile containing your repository to your settings.xml**.
- Paste the code snippet you copied right underneath the <servers> tags. Make sure the <profiles> tags are also nested inside the <settings> tags
- Finally, paste the XML code snippet from **Step 6: (Optional) Set a mirror in your settings.xml...** right underneath the <profiles> tags
- Save the settings.xml file

```
src > main > webapp > settings.xml
  8    </servers>
  9    <profiles>
 10      <profile>
 11        <id>nextwork-nextwork-devops-cicd</id>
 12        <activation>
 13          <activeByDefault>true</activeByDefault>
 14        </activation>
 15        <repositories>
 16          <repository>
 17            <id>nextwork-nextwork-devops-cicd</id>
 18            <url>https://nextwork-022687721466.d.codeartifact.us-
 19          </repository>
 20        </repositories>
 21      </profile>
 22    </profiles>
 23    <mirrors>
 24      <mirror>
 25        <id>nextwork-nextwork-devops-cicd</id>
 26        <name>nextwork-nextwork-devops-cicd</name>
 27        <url>https://nextwork-022687721466.d.codeartifact.us-east
 28        <mirrorOf>*</mirrorOf>
 29      </mirror>
 30    </mirrors>
 31    </settings>

TERMINAL    bash - nextwork-web-project

[ec2-user@ip-172-31-18-246 nextwork-web-project]$ export CODEARTIFACT_AUTH
_TOKEN=`aws codeartifact get-authorization-token --domain nextwork --domai
n-owner 022687721466 --region us-east-2 --query authorizationToken --outpu
t text`
[ec2-user@ip-172-31-18-246 nextwork-web-project]$
```

Compile your project and verify the CodeArtifact integration

- In your VS Code terminal, run pwd to check that you're in the root directory of your nextwork-web-project
- If you're not at the root directory, run cd nextwork-web-project to get there!
- Next, we'll **compile** your project
- Run the Maven compile command, which uses the settings.xml file we just configured: **mvn -s settings.xml compile**
- If the compilation is successful and dependencies are downloaded from CodeArtifact, you'll see a BUILD SUCCESS message at the end of the Maven output.

```
Downloaded from nextwork-nextwork-devops-cicd: https://nextwork-022687721466.d.codeartifact.
us-east-2.amazonaws.com/maven/nextwork-devops-cicd/com/google/collections/google-collections
/1.0/google-collections-1.0.jar (640 kB at 408 kB/s)
[INFO] No sources to compile
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time: 51.012 s
[INFO] Finished at: 2026-04-22T17:04:40Z
[INFO] Final Memory: 11M/28M
[INFO] ------------------------------------------------------------------------
[ec2-user@ip-172-31-18-246 nextwork-web-project]$
```

Create a CodeBuild Project

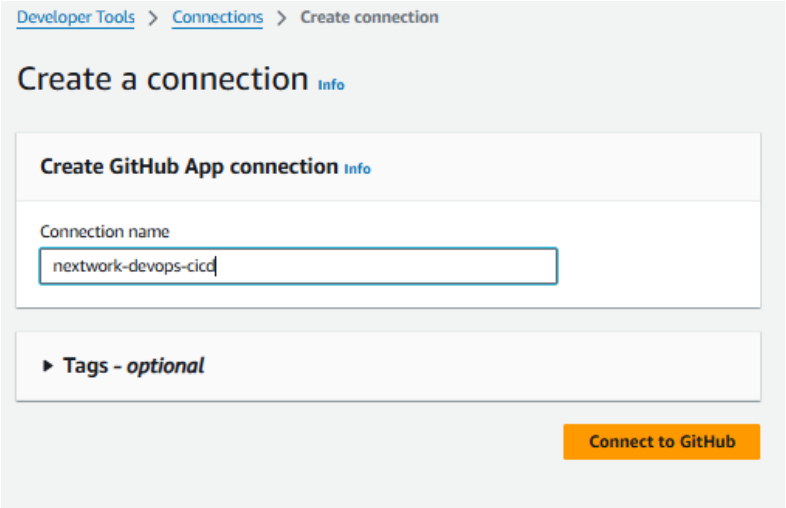
- Log in to your AWS Management Console.
- In the top search bar, type CodeBuild.
- Select **CodeBuild** from the dropdown menu under **Services**.
- In the CodeBuild dashboard, find the left navigation menu.
- Select **Build projects**.

Configure Your Build Project

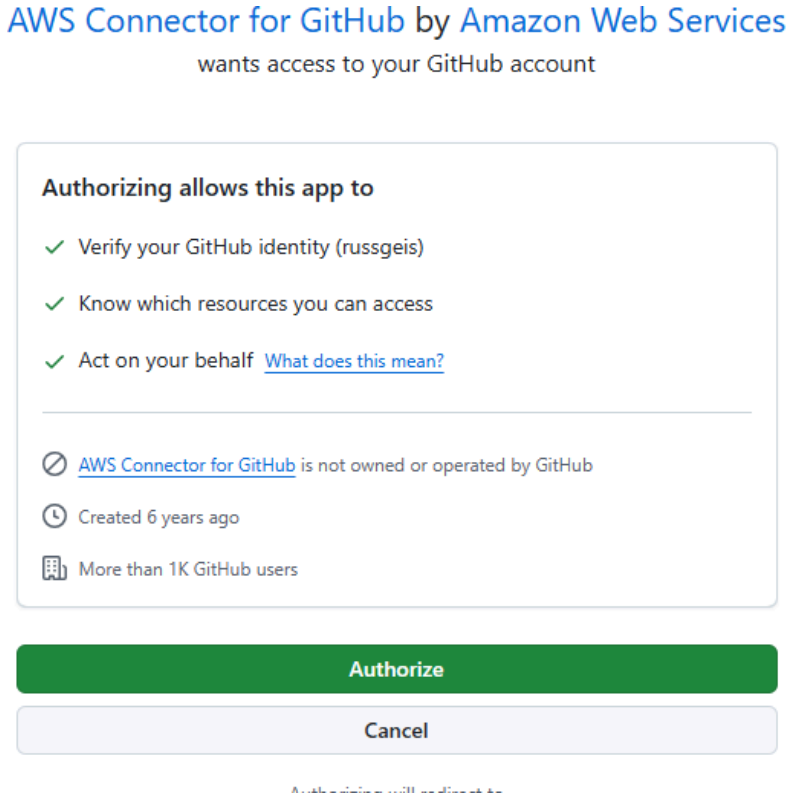
- On the **Create build project** page, scroll to the **Project configuration** section.
- Under **Project name**, enter nextwork-devops-cicd.
- Under **Project type**, make sure **Default project** is selected.
- Scroll down to the **Source** section
- Under **Source provider**, select **GitHub**

Let's get started! Connecting to GitHub is crucial for CodeBuild to access your project's code.

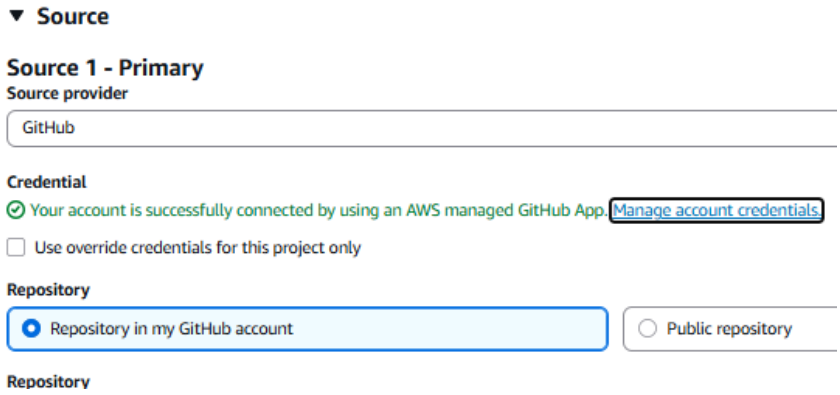
- In the **Source** section, under **Credential**, you might see the message You have not connected to GitHub. Manage account credentials.
- Click on **Manage account credentials**.
- Ensure **GitHub App** is selected for **Credential type**.
- Select **create a new GitHub connection**
- On the **Create connection** page, under **Connection details**, enter nextwork-devops-cicd as the **Connection name**
- Click **Connect to GitHub**.



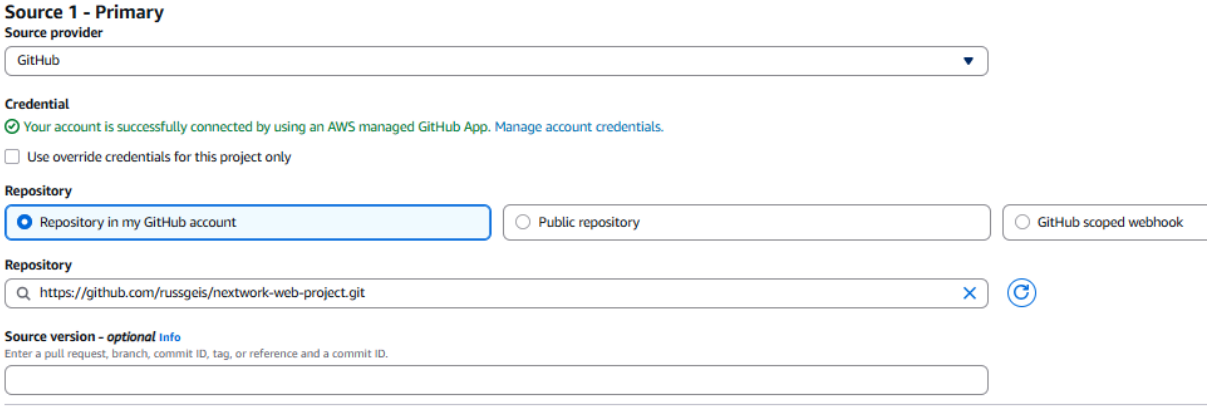
- You will be taken to GitHub to authorize the **AWS Connector for GitHub** application.
- Select your GitHub user account where your repository is located.



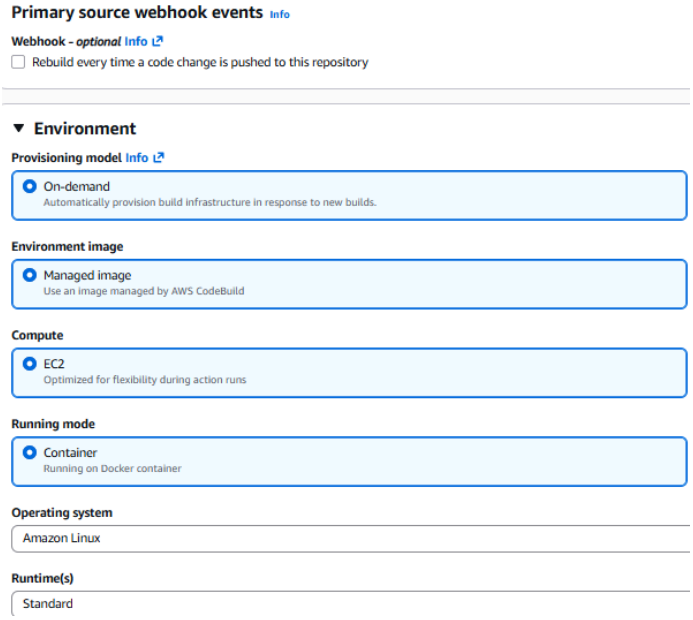
- Select **Select**.
- After authorization on GitHub, you'll get taken back to the AWS console.
- Under **GitHub Apps**, you'll see that your GitHub username is an option now!
- Select your GitHub username
- Click **Connect**.
- Now, back in the **Create build project** page, in the **Source** section, you should see a success message in green: "Your account is successfully connected by using an AWS managed GitHub App."



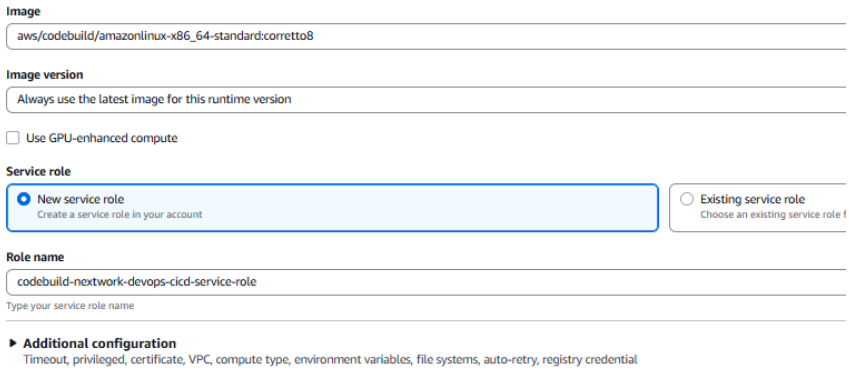
- You can now select your GitHub repository nextwork-web-project as the source. May need to add the URL to it



- Scroll down to the **Primary source webhook events** section.
- **Untick** the **Webhook** checkbox that says "Rebuild every time a code change is pushed to this repository."* Scroll down to the **Environment** section.
- Under **Compute**, for **Provisioning model**, choose **On-demand**.
- For **Environment image**, choose **Managed image**
- For **Compute type**, choose **EC2**
- Under **Environment**, for **Operating system**, select **Amazon Linux**.
- For **Runtime(s)**, select **Standard**



- For **Image**, choose aws/codebuild/amazonlinux-x86_64-standard:corretto8
- Keep **Image version** as **Always use the latest image for this runtime version**.
- Under **Service role**, select **New service role**.



Now, let's define how CodeBuild will actually build your application using a buildspec.yml file.

- Scroll down to the **Buildspec** section.
- Under **Buildspec format**, select **Use a buildspec file**.
- Leave **Buildspec name** as default buildspec.yml
- Scroll down to the **Batch configuration** section

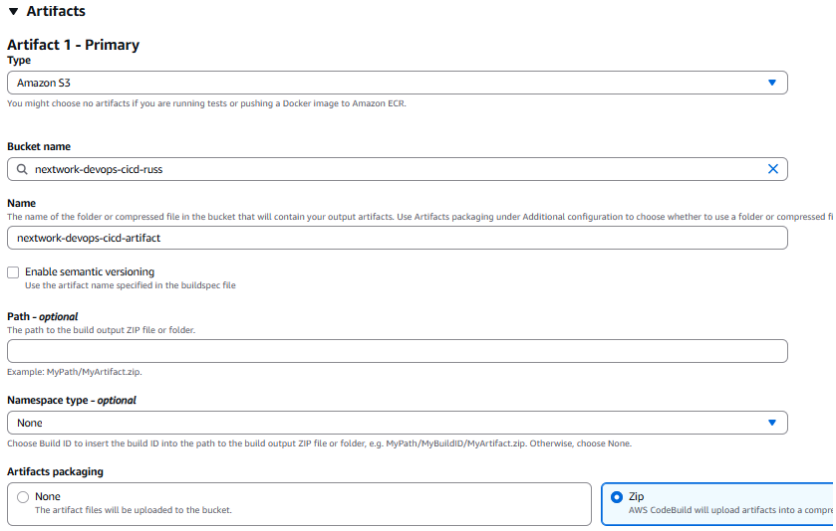
Configure Build Artifacts

- Scroll down to the **Artifacts** section. We need to configure where CodeBuild will store the **build artifacts**
- For **Type**, select **Amazon S3**.
- Let's head to the S3 console. In the AWS Management Console search bar, type S3 and select **S3** from the dropdown menu under **Services**.
- Make sure you're still in the same region where you set up the CodeBuild build project.
- Leave all other settings as default.
- Click **Create bucket** at the bottom of the page.

Back in CodeBuild tab: If you just created the S3 bucket, it might not show as an option in the CodeBuild project configuration.

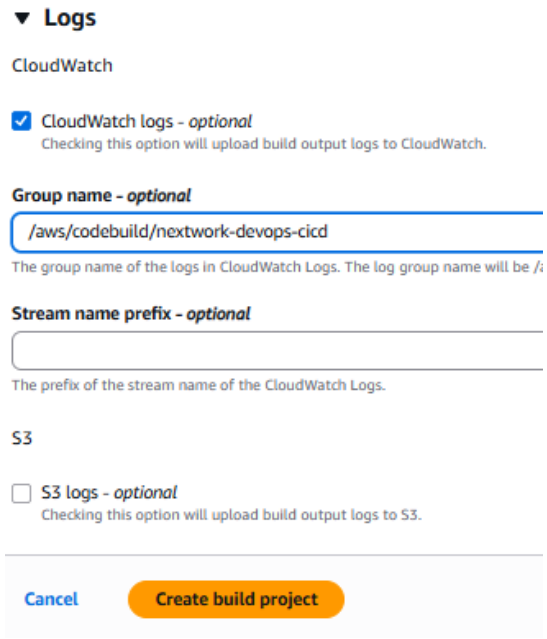
- **Refresh** the build project page in your browser - you might need to configure your build project again. Challenge yourself to re-do the setup with less guidance!
- In your CodeBuild project, head back to the **Artifacts** section.
- For **Type**, select **Amazon S3**.
- For **Bucket name**, choose your newly created bucket nextwork-devops-cicd from the dropdown.

- In **Name**, enter nextwork-devops-cicd-artifact. This names our artifact, so it's easy to spot it in the S3 bucket.
- For **Artifacts packaging**, select **Zip**



Configure CloudWatch Logs

- Scroll down to the **Logs** section.
- Make sure **CloudWatch logs** is checked.
- In **Group name**, enter /aws/codebuild/nextwork-devops-cicd



Wooooooop woop! You've configured logs for your CodeBuild project. You're all set to create your project and run your first build!

- Scroll to the bottom of the page and click **Create build project**